

# Week 3 (CS 418 @ UIC)

---

# Week 3 Slide Deck

## Wrangling, Filtering; Formats

Jack Bandy 2026



# Topic Title Placeholder

CS 418 · Week 3 · Quincy









# Data Formats

---

# How Do We Store This Data?



# Spreadsheet??

- You've probably seen lots of spreadsheets
- Grid with cells, rows, columns (and sheets/tabs)
- Excel, Google Sheets, Numbers, LibreOffice Calc, etc.
- Convenient for manual entry, review, sharing
- But data scientists don't keep data in spreadsheets...

|   | A            | B                     | C                | D        | E              |
|---|--------------|-----------------------|------------------|----------|----------------|
| 1 | name         | company               | street           | city     | phone          |
| 2 | Tyler Durden | Paper Street Soap Co. | 537 Paper Street | Bradford | (288) 555-0153 |

# Why Not Spreadsheets?

- Formulas and transformations are hidden inside cells
- Data types, missing values can be ambiguous
- Auto-formatting can corrupt values (e.g., gene name “SEPT2” becomes a date)<sup>[6]</sup>
- Errors can go undetected for years without automated testing <sup>[7,8]</sup>
- No separation between data, logic, and presentation
- Version control can be more difficult
  - Sharing files -> accidental overwrites, conflicts
- Larger datasets become slow



- Comma-separated values
- Plain-text table: one row per record, one delimiter between fields
- Often used for spreadsheets, exports, and simple datasets
- Easy to inspect
- types and hierarchy are usually implicit

```
1 name,company,product,street,city,postalCode,phone
2 Tyler Durden,Paper Street Soap Co.,All Natural Handmade,537 Paper Street
```

- Tab-separated values
- Plain-text table like CSV, but fields are separated by tabs
- Still flat: hierarchy and data types need outside context

```
1 name      company product street  city      postalCode  phone
2 Tyler Durden    Paper Street Soap Co.  All Natural Handmade    537 Pa
```



# JSON

- JavaScript Object Notation
- Text format for data interchange
- Built from objects, arrays, strings, numbers, booleans, and null
- Common for web APIs and configuration files

```
1 {  
2   "name": "Tyler Durden",  
3   "company": "Paper Street Soap Co.",  
4   "product": "All Natural Handmade",  
5   "address": {  
6     "street": "537 Paper Street",  
7     "city": "Bradford",  
8     "postalCode": "19808"  
9   },  
10  "phone": "(288) 555-0153"  
11 }
```

- Extensible Markup Language
- Nested tags represent elements and attributes
- Verbose, but widely used by older document systems
- E.g. Excel uses xml under the hood

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <person>
3   <name>Tyler Durden</name>
4   <company>Paper Street Soap Co.</company>
5   <product>All Natural Handmade</product>
6   <address>
7     <street>537 Paper Street</street>
8     <city>Bradford</city>
9     <postalCode>19808</postalCode>
10  </address>
11  <phone>(288) 555-0153</phone>
12 </person>
```

- YAML Ain't Markup Language
- Human-readable format based on indentation
- Supports mappings, lists, scalars, and comments
- Common for configuration files and data pipelines

```
1 name: Tyler Durden
2 company: Paper Street Soap Co.
3 product: All Natural Handmade
4 address:
5   street: 537 Paper Street
6   city: Bradford
7   postalCode: 19808
8 phone: (288) 555-0153
```

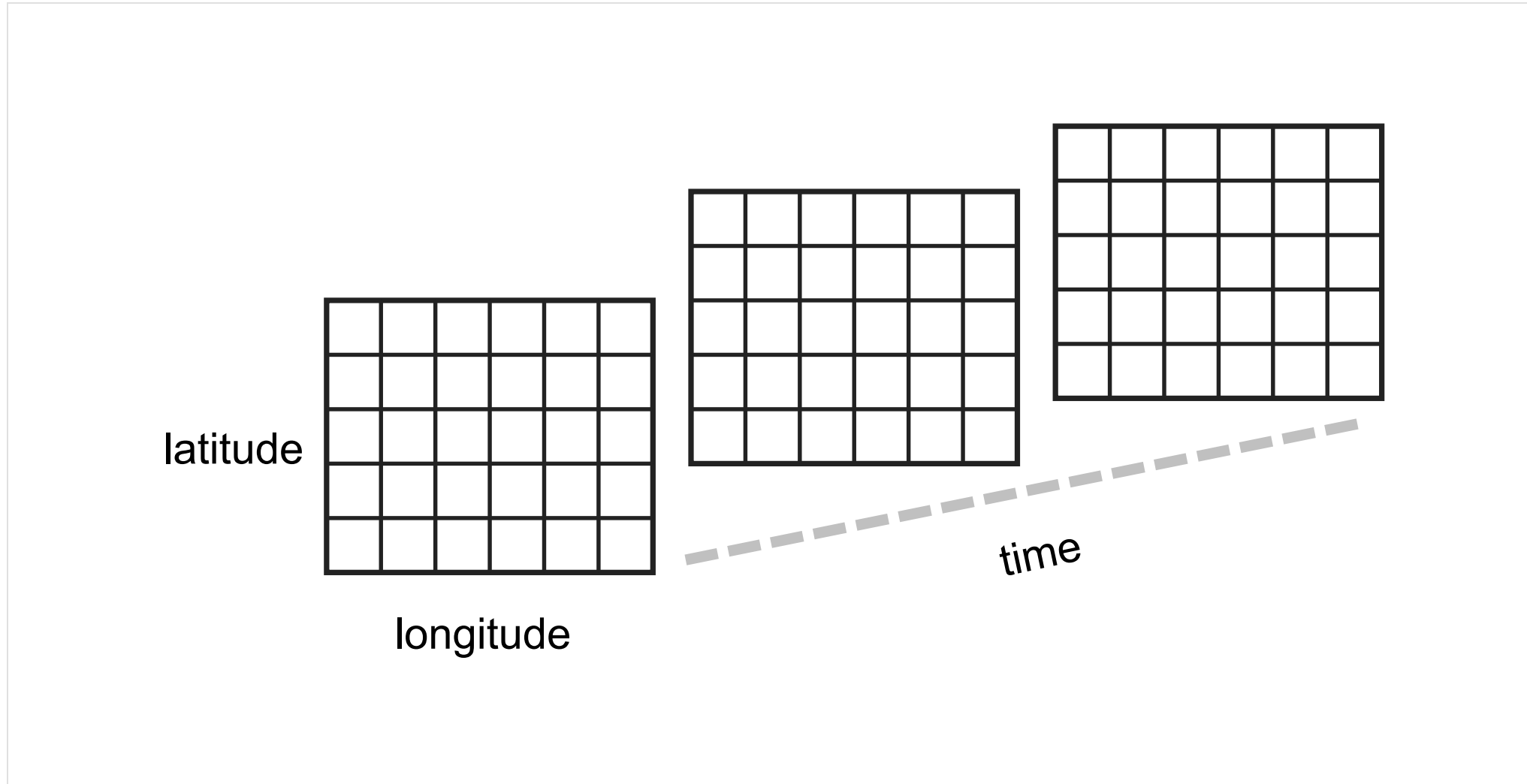


# Parquet

- Binary columnar storage format
- Efficient for large datasets
- Stores schema and data types with the data
- Common in Spark, DuckDB, Polars, and “data lakes”

```
1 message business_card {
2   required binary name (STRING);
3   required binary company (STRING);
4   required binary product (STRING);
5   required binary street (STRING);
6   required binary city (STRING);
7   required binary postalCode (STRING);
8   required binary phone (STRING);
9 }
10
11 row 1:
12 Tyler Durden | Paper Street Soap Co. | All Natural Handmade |
13 537 Paper Street | Bradford | 19808 | (288) 555-0153
```

# What About More Complex Data?



- Network Common Data Form
- Binary, self-describing format for array-oriented scientific data
- Stores dimensions, variables, and metadata together
- Better than CSV or JSON for gridded data with coordinates
- Common in scientific workflows (e.g. climate, ocean, weather)
- Harder to quickly inspect than plain text formats
  - (Data stored as binary)

```
1 dimensions:
2   time = unlimited
3   lat = 180
4   lon = 360
5
6 variables:
7   float temperature(time, lat, lon)
8     temperature:units = "K"
9     temperature:long_name = "air temperature"
10  float lat(lat)
11  float lon(lon)
```

*A simplified example of what a NetCDF file can contain.*

# Format Summary

| Format      | Best fit                                           | Short example                                                                                     |
|-------------|----------------------------------------------------|---------------------------------------------------------------------------------------------------|
| CSV         | Flat tables, spreadsheet exports, simple datasets  | <code>name,company,phone</code><br><code>Tyler Durden,Paper Street Soap Co.,(288) 555-0153</code> |
| TSV         | Flat text tables where commas may appear in fields | <code>name company phone</code>                                                                   |
| JSON        | APIs, nested records, web data                     | <code>{"name":"Tyler Durden","city":"Bradford"}</code>                                            |
| XML         | Document-like data with tags and attributes        | <code>&lt;name&gt;Tyler Durden&lt;/name&gt;</code>                                                |
| YAML        | Human-edited configuration and pipeline settings   | <code>name: Tyler Durden</code><br><code>city: Bradford</code>                                    |
| NetCDF      | Labeled multi-dimensional scientific arrays        | <code>temperature(time, lat, lon)</code>                                                          |
| Parquet     | Typed, compressed analytics data                   | <code>name: STRING</code><br><code>city: STRING</code>                                            |
| Spreadsheet | Small, early-stage data; prototypes; manual entry  | <i>(Excel, Google Sheets, Numbers)</i>                                                            |



# Sources

1. GitHub source: <https://github.com/jackbandy/data-science-fun/blob/main/docs/slides/week3.md>.
2. Last modified and compiled June 24, 15:02h Central (Chicago) time.
3. NetCDF reference: Unidata netCDF documentation, <https://www.unidata.ucar.edu/software/netcdf/>.
4. LearningDS chapter 14, “Web Data,” netCDF discussion: [https://learningds.org/ch/14/web\\_netCDF.html](https://learningds.org/ch/14/web_netCDF.html).
5. Format examples adapted from Wikipedia and project documentation: [JSON](#), [XML](#), [Comma-separated values](#), [YAML](#), [Tab-separated values](#), [Spreadsheet](#), and [Apache Parquet](#).
6. Tyler Durden business card image: Wikimedia Commons remake by Michaelpreid, modified, [CC BY-SA 4.0](#), [https://commons.wikimedia.org/wiki/File:Tyler\\_Durden\\_Business\\_Card.png](https://commons.wikimedia.org/wiki/File:Tyler_Durden_Business_Card.png).
7. Ziemann M., Eren Y., El-Osta A., “Gene name errors are widespread in the scientific literature,” *Genome Biology* 17, 177 (2016): <https://link.springer.com/article/10.1186/s13059-016-1044-7>.
8. European Spreadsheet Risks Interest Group (EuSpRIG), Horror Stories — a catalogue of real-world spreadsheet errors in business, government, and research: <https://eusprig.org/research-info/horror-stories/>.
9. Inman, Phillip, “The Reinhart-Rogoff error — or how not to Excel at economics,” *The Conversation*, 2013: <https://theconversation.com/the-reinhart-rogooff-error-or-how-not-to-excel-at-economics-13646>.
10. Slides developed using materials from [Elena Zheleva](#) and [Gonzalo Bello Lander](#), the Berkeley DS 100 team, Marine Carpuat, and Brian Ziebart.
11. Slide deck built with [Quarto](#) revealjs.
12. Title font is Big Shoulders; Body font is [Libre Franklin](#).